```
            .Select(x => new RecipeViewModel
            {
                Id = x.RecipeId,
                Name = x.Name
            })
            .SingleOrDefault();
    }
}
```

❶ An EF Core DbContext is injected in the constructor.

❷ Uses the DbSet<Recipes> property to load recipes and creates a RecipeViewModel

Writing unit tests for this class is a bit of a problem. Unit tests should be fast, repeatable, and isolated from other dependencies, but you have a dependency on your app's `DbContext`. You probably don't want to be writing to a real database in unit tests, as it would make the tests slow, potentially unrepeatable, and highly dependent on the configuration of the database—a failure on all three requirements!

NOTE Depending on your development environment, you may want to use a real database for your integration tests, despite these drawbacks. Using a database like the one you'll use in production increases the likelihood that you'll detect any problems in your tests. You can find an example of using Docker to achieve this in Microsoft's "Testing ASP.NET Core services and web apps" documentation at http://mng.bz/zxDw.

Luckily, Microsoft ships two in-memory database providers for this scenario. Recall from chapter 12 that when you configure your app's `DbContext` in Program.cs, you configure a specific database provider, such as SQL Server:

```
builder.Services.AddDbContext<AppDbContext>(options =>
   options.UseSqlServer(connectionString);
```

The in-memory database providers are alternative providers designed only for testing. Microsoft includes two in-memory providers in ASP.NET Core: